

Homework 1

TCP Congestion Control

1. *Why does TCP repeatedly increase the congestion window until it detects a problem (timeout or triple duplicate ACKs), then decrease it instead of finding an optimal maximum then staying there?*

Because the maximum transmission rate is variable, due to the network being used by other TCP senders. By staying fixed, if bandwidth clears up, we risk not using it, and if the opposite happens, we will likely incur segment losses.

2. *Why are there two distinct phases, one with an exponential increase in congestion window, and one with a linear increase.*

This is because an exponential increase is preferable at the start of the increase, since the sending rate is doubled for each successful MSS ack. When we reach a threshold, we risk going way over an acceptable rate by continuing to double, so we switch to linear (congestion avoidance) to probe the link's capacity.

3. *During congestion avoidance, the congestion window increases linearly, but upon receiving triple duplicate acks, why does the congestion window halve instead of decreasing linearly.*

This is because we enter the "fast recovery" phase. The network seems to have been congested since a segment was lost, so we step back.

Repeat

a. *Visit the Go-Back-N Java applet at this website. Try it out and get familiar with how it works then, using the applet, answer the following questions:*

1. *Have the source send five packets and then pause the animation before any of the five packets reach the destination. Then kill the second packet and resume the animation. Describe what happens and why.*

All the packages after the second package that reach the receiver are dropped - it is expecting the second package. The sender receives the same ack multiple times and starts a timeout before trying to send the missing package and all subsequent packages again.

2. *Repeat the experiment but now let the second packet reach the destination and kill the second acknowledgement. Describe again what happens and why.*

All packages are marked as received on the sender. This is because the receiver would not say that package 3 was acknowledged if it had not already acknowledged package 2. This means the sender can assume the package 2 ack was just lost in transmission.

b. Visit the Selective Repeat Java applet at this website and repeat those questions.

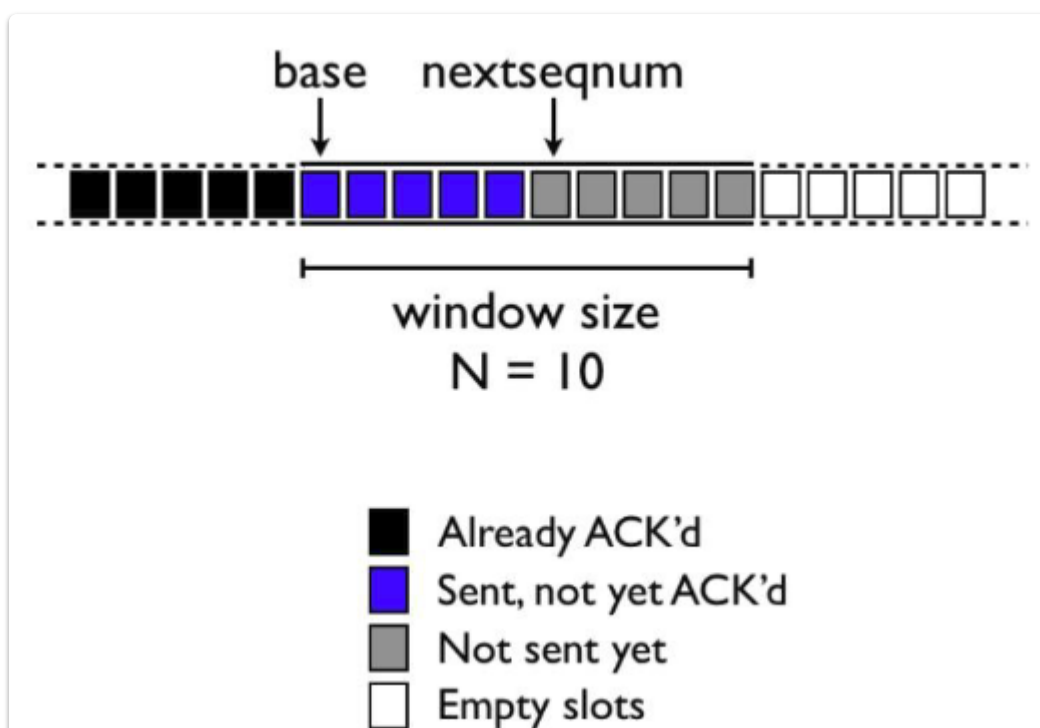
1. *Have the source send five packets and the pause the animation before any of the five packets reach the destination. Then kill the second packet and resume the animation. Describe what happens and why.*

The receiver marks the second packet as missing, but sends back ACKs for each specific package succeeding it, letting the sender know it only needs to resend the second one. This behavior gives the algorithm its name.

2. *Repeat the experiment but now let the second packet reach the destination and kill the second acknowledgement. Describe again what happens and why.*

The receiver has marked each package as received, but since the sender does not know this, it resends the second package. Because the sender has already ACK'd this package, it does not change its data and sends back a DUPACK, which finally satisfies the sender.

Sliding window

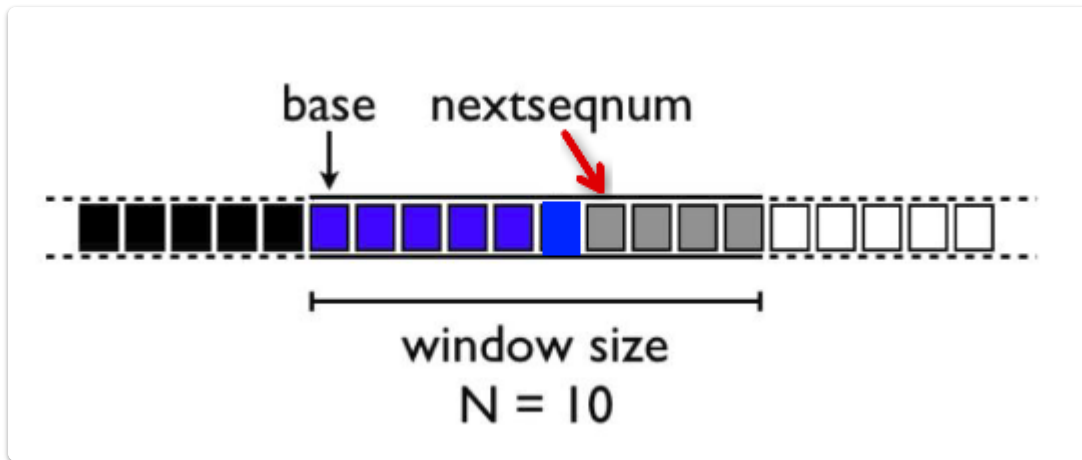


Using the figure above, answer a few questions related to the sliding window

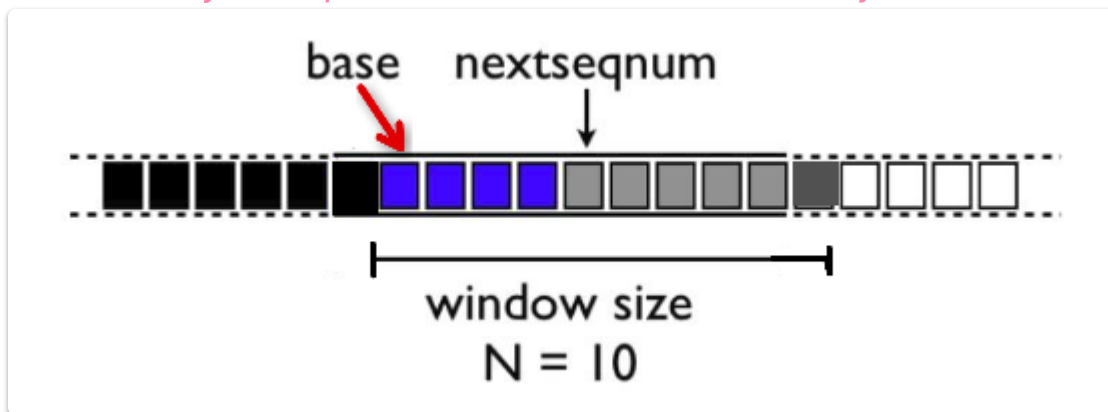
principle. The figure is imitated from the section on Go-back-n in the book.

a) Draw the figure again showing where the base, nextseqnum are, and how the box colors change after each of the following events:

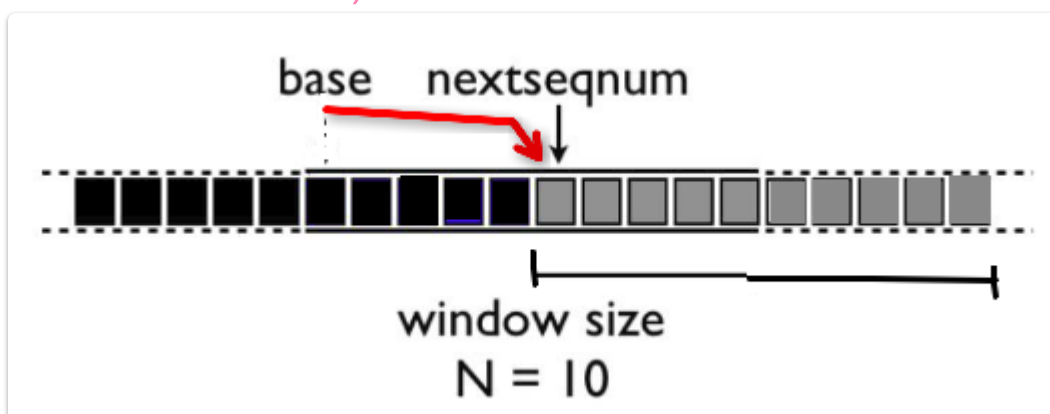
1. One new packet has been sent from the transmitter, but an ACK has not been received.



2. The ACK for the packet at the base is received by the transmitter.



2. The transmitter receives an ACK for all the sent packets (assume all ACKs are received in order)



b. How many bits would you need to indicate the sequence number in each packet with the following protocols? (Hint: Consider the window size.)
Answer with formulas, and make sure to avoid ambiguity.

1. *Go-Back-N*

$\log_2(N)$ rounded up, where N is the window size.

(When base (0) has been used and ACK'd by sender and receiver, it *can* be reused for the nextseqnum.)

2. *Selective repeat*

At least $\log_2(N * 2)$ rounded up, where N is the window size.

(Window size (N) must be equal to - or less than - half of the size of the sequence number protocol ($N * 2$))

In reality the TCP sequence number field is 32 bits long.